Software Re-Engineering

Lecture 20

Migration to Object oriented platforms

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Readings

Chapter 2 to 5, Object-Oriented Reengineering Patterns by Serge Demeyer

Patterns

Patterns turn out to be a compact way to communicate best practice

Reengineering patterns codify and record knowledge about modifying legacy software: they help in diagnosing problems and identifying weak nesses which may hinder further development of the system, and they aid in finding solutions which are more appropriate to the new requirements.

Design patterns vs reengineering patterns

Design patterns have to do with choosing a particular solution to a design problem, whereas reengineering patterns have to do with discovering an existing design, determining what problems it has, and repairing these problems.

As a consequence, reengineering patterns have more to do with the process of discovery and transformation

Whereas a design pattern presents a solution for a recurring design problem, a reengineering pattern presents a solution for a recurring reengineering problem. The artifacts produced by reengineering patterns are not necessarily designs. They may be as concrete as refactored code

Re engineering patterns

- Setting Direction
- First contact
- Initial understanding
- Detailed model capture

Setting Direction

- Agree on Maxims
- Appoint a Navigator
- Speak to the Round Table
- Most Valuable First
- Fix Problems, Not Symptoms
- If It Ain’t Broke, Don’t Fix It
- Keep It Simple

Initial Understanding

- Analyze the Persistent Data
- Speculate about Design
- Study the Exceptional Entities

First Contact

- Chat with the Maintainers
- Read all the Code in One Hour
- Skim the Documentation
- Interview During Demo
- Do a Mock Installation

Detailed Model Capture

- Tie Code and Questions Refactor to Understand
- Step Through the Execution
- Look for the Contracts
- Learn from the Past

Setting Direction

- Many different factors might affect a re-engineering project
- Technical, ergonomic, economic, and political considerations make it hard to establish and maintain focus
- Hard to set priorities between the many problems of the legacy software
- Danger of focusing on interesting parts rather than what’s good for the system

Setting Direction Patterns

- **Agree on Maxims**

You should Agree on Maxims in order to establish a common understanding within the re-engineering team of what is at stake and how to achieve it.

Maxims are rules of conduct that can help steer a project that is pulled in many directions.

Goldberg and Rubin [GR95] give numerous examples of maxims, such as “Everyone is responsible for testing and debugging” and “You cannot do it right the first time.”

Setting Direction Patterns

- **Appoint a Navigator**

You should Appoint a Navigator to maintain the architectural vision.

The architecture of any system tends to degrade with time as it becomes less relevant to new, emerging requirements.

The challenge of a reengineering project is to develop a new architectural vision that will allow the legacy system to continue to live and evolve for several more years.

Setting Direction Patterns

- **Speak to the Round Table**

Everyone should Speak to the Round Table to maintain team awareness of the state of the project.

Regular, focussed, round table meetings can achieve the goal of keeping team members synchronized with the current state of affairs.

Round table meetings should be brief, but everyone must be required to contribute.

A simple approach is to have everyone say what they have done since the last meeting, what they have learned or perhaps what problems they have encountered, and what they plan to do until the next meeting.

Setting Direction Patterns

- **Most Valuable First**

A legacy system may suffer from a great number of problems, some of which are important, and others which may not be at all critical for the customer’s business.

By focussing on the most valuable parts first, you increase the chance that you will identify the right issues at stake, and that you will be able to test early in the project the most important decisions, such as which architecture to migrate to, or what kind of flexibility to build into the new system.

Setting Direction Patterns

- **Fix problems, not symptoms**

Address the source of a problem, rather than particular requests of your stakeholders.

> If the code of a legacy component is basically stable, and problems mainly occur with changes to clients, then the problem is likely to be with the interface to the legacy component, rather than its implementation, no matter how nasty the code is. In such a case, you should consider applying Present the Right Interface to just fix the interface.
>
> If the legacy component is largely defect-free, but is a major bottle neck for changes to the system, then it should probably be refactored to limit the effect of future changes. You might consider applying Split Up God Class to migrate towards a cleaner design.

Setting Direction Patterns

- **If it ain’t broke, don’t fix it**

Anytime you “fix” something, you also risk breaking something else in the system. You also risk wasting precious time and effort on marginal issues.

Setting Direction Patterns

- **Keep it simple**

An important reengineering goal is to accommodate future change. But too much flexibility will make the new system so complex that you may actually impede future change.

First contract

- All the patterns in this cluster can be applied to the very early stages of a re-engineering project: you're facing a system that is completely new for you, and within a few days you must determine whether something can be done with it and present a plan how to proceed.
- Making such an initial assessment is difficult, however, because you quickly need accurate results while considering the long-term effects of your decisions.

Time is scarce. Wasting time early on in a project has severe consequences later on. This is especially relevant during reverse engineering, because there you feel uncertain and then it is tempting to start an activity that will keep you busy for a while instead of address ing the root of the problem.

Consequently, consider time as your most precious resource. Therefore, defer all time-consuming activities until later and use the first days of the project to assess the feasibility of the project's goals.

All patterns in this cluster are meant to quickly identify the opportunities and risks for your project and as such will help you set the overall direction of the project.

First impressions are dangerous. Making important decisions based on incomplete knowledge implies that there is a chance you will make the wrong decision. There is no way to avoid that risk during your first contact with a system, however you can minimize its impact if you always double-check your sources.

Chat with the Maintainers

Learn about the historical and political context of your project through discussions with the people maintaining the system.

Documentation, if present, typically records decisions about the solution, not about the factors which have influenced that solution. Consequently, the important events in the history of the system (i.e., its historical context) are rarely documented.

Persons working with the system might mislead you. Sometimes people will deliberately deceive you, especially when they are responsible for the problematic parts of the system or when they want to protect their jobs.

Hints

Here are some questions that may help you while discussing with the maintainers.

What was the easiest bug you had to fix during the last month? And what was the most difficult one? How long did it take you to fix each of them? Why was it so easy or so difficult to fix that particular bug?

How does the maintenance team collect bug reports and feature requests? Who decides which request gets handled first? Who decides to assign a bug report or feature request to a maintainer? Are these events logged in some kind of database? Is there a version or configuration management system in place?

These questions help to understand the organization of the maintenance process and the internal working habits of the maintenance team. As far as the political context concerns, it helps to assess the relationship within the team (task assignment) and with the end users (collection of bug reports).

Read all the Code in One Hour

Assess the state of a software system by means of a brief, but intensive code review.

- Grant yourself a reasonably short amount of study time (i.e., approximately one hour) to read the source code. Make sure that you will not be disturbed (unplug the telephone and disconnect your e-mail) and take notes sparingly to maximize the contact with the code.
- After this reading session, produce a short report about your findings, including
  - ❑ a general assessment of whether re-engineering seems feasible and why (not);
  - ❑ entities which seem important (i.e., classes, packages, ⋯);
  - ❑ suspicious coding styles discovered (i.e., “code smells” [FBB+99]);
  - ❑ parts which must be investigated further (i.e., tests).
- Keep this report short, and name the entities like they are mentioned in the source code.

After you **Read all the Code in One Hour** you should Do a Mock Installation to evaluate the suitability for re-engineering. You may complement your findings if you **Skim the Documentation** and carry out an **Interview During Demo** to maximize your chances of getting a coherent view of the system. Before actually making a decision on how to proceed with the re-engineering project, it is probably worthwhile to Chat with the Maintainers once more.

Skim the Documentation

Intent Assess the relevance of the documentation by reading it in a limited amount of time.

- Prepare a list summarizing those aspects of the system that seem interesting for your reengineering project.
- Then, match this list against the documentation and meanwhile make a crude assessment of how up to date the documentation seems.

Finally, summarize your findings in a short report, including

- steer the reading process to match your main interest. For instance, if you want insight into the original system requirements then you should look inside the system specification
- knowledge about which features are actually implemented should be collected from the end-user manual or tutorial notes.
- Check whether the documentation is outdated with respect to the actual system. Always compare version dates with the date of delivery of the system and make note of those parts that you suspect are unreliable.

Interview During Demo

Obtain an initial feeling for the appreciated functionality of a software system by seeing a demo and interviewing the person giving the demo.

- do the demonstration several times with different persons giving the demo.
- This way you will see variations in what people find important and you will hear different opinions about the value of the software system.

Do a Mock Installation

Check whether you have the necessary artefacts available by installing the system and recompiling the code.

- Log all small failures you encounter during the build and installation process and the way you solved them, because this will tell you about the configuration of the system and its dependencies on libraries, frameworks and patches.

Initial Understanding

- When developing your initial understanding of a software system, incorrect information is your biggest concern. Therefore these patterns rely mainly on source-code because this is the only trustworthy information source.
- In principle, there are two approaches for studying source-code: one is top-down, the other is bottom-up. In practice, every reverse engineer ing approach must incorporate a little bit of both, still it is worthwhile to make the distinction. With the top-down approach, you start from a high level representation and verify it against the source-code (as for instance described in Speculate about Design). In the bottom-up approach, you start from the source-code, filter out what's relevant and cast the relevant entities into a higher-level representation

Analyze the Persistent Data

Learn about objects that are so valuable they must be kept inside a database system

Speculate about Design

Progressively refine a design against source code by checking hypotheses about the design against the source code.

Use your development expertise to conceive a hypothetical class diagram representing the design.

Refine that model by verifying whether the names in the class diagram occur in the source code and by adapting the model accordingly.

Repeat the process until your class diagram stabilizes.

Study the Exceptional Entities

Identify potential design problems by collecting measurements and studying the exceptional values.

Measure the structural entities forming the software system (i.e., the inheritance hierarchy, the packages, the classes and the methods) and look for exceptions in the quantitative data you collected.

Verify manually whether these anomalies represent design problems

Detailed Model Capture

- build up a detailed model of those parts of the system that will be important for your reengineering effort.
- It is important to understand that the intent of this pattern is not to improve the code base itself, but only to improve your understanding.
- Since the source code gives you only a very static view of the class hierarchy, it is useful to Step Through the Execution to learn what objects are instantiated and run time and how they interact.

Tie Code and Questions

Keep the questions and answers concerning your reengineering activities synchronized with the code by storing them directly in the source files

In principle there are two ways to annotate the code.

Comment-based Annotations.

This approach uses the commenting conventions of the programming language and as such is better suited for a text-oriented environment. A few conventions are needed to distinguish the normal comments from the annotations.

```
/* #to: John #by: SD #on: 3/12/99 *****
Screws up when we have nested IFs. */
```

Method-based annotations.

This approach exploits the possibility to query which method invokes a given method, a feature provided by many of today's programming environments. The idea is to declare a global method accepting a few strings as an argument and having an empty method body. Each time you want to annotate a particular piece of code, you invoke that method passing your annotations as a parameter.

```
this.annotateCode("#to: John #by: SD #on: 3/12/99",
"Screws up when we have nested IFs.");
```

Refactor to Understand

Iteratively refactor a part of a software system in order to validate and reflect your understanding of how it works

- Iteratively rename and refactor the code to introduce meaningful names and to make sure the structure of the code reflects what the system is actually doing.
- Run regression tests after each change if they are available, else compile often to check whether your changes make sense.
- Decide what to do with the code after you have refactored it.

Your primary goal here is to understand the system, not to improve the code. The changes you make to the code should therefore be treated as “experiments” to test your understanding of the code. As a consequence, you should make a copy of the code before you start.

The following guidelines will help you to find out where and how to apply these refactorings to improve the readability of the code.

Rename attributes to convey roles.
To find out about their roles, look at all the attribute accesses (including invocations of accessors methods). Afterwards, rename the attribute and its accessors according to its role, update all references and re-compile the system.

Rename methods to convey intent.
To retrieve the intent of a method that does not have an intention revealing name, investigate all invo cations and attribute uses, and deduce the method's responsibility. Afterwards, rename the method according to its intent, update all invocations and re-compile the system.

Rename classes to convey purpose.
Remove duplicated code.

Replace condition branches by methods.

If you encounter conditions with large branches, extract the leaves as new (private) methods. To name these methods, study the condition until you understand it well enough to choose an intention revealing name.

Refactor method bodies to a consistent level of abstraction.

Long method bodies with comments separating blocks of code violate the rule of the thumb that all statements in a single method body should have the same level of abstraction. Refactor such code by introducing a new (private) method for each separated block of code; name the method after the intent recorded in the comment.

Step Through the Execution

Understand how objects in the system collaborate by stepping through examples in a debugger.

- It is too time-consuming to step through every single statement of a run ning system.
- The assumption here is that you are focussed on some specific aspect of the system that is difficult to understand.
  - ❑ Set breakpoints to interrupt execution when the system enters the code you are interested in.
  - ❑ Change the internal state of the objects to see how alternative execution paths are triggered.

Look for the Contracts

Infer the proper use of a class interface by studying the way clients currently use it.

Look for common programming idioms that expose the way clients make use of the class interface.
Generalize your observations in the form of contracts, i.e., explicit declarations of what a class expects from its clients.

Learn from the Past

Obtain insights into the design by comparing subsequent versions of the system.

- Use the metrics or configuration management tool to find entities where functionality has been removed, because such entities are a sign of a consolidating design.
- Also, look for entities which change often as these may point you to an unstable part of the design.

Unstable design.

If you detect repeated growth and refactoring in the same portion of the system, that should be a sign that the design is unstable. It may indicate opportunities to redesign that portion of the system to bet ter accommodate the kinds of changes and extensions that habitually take place.

Legacy systems often do not have test procedures defined.

Changing parts of a system without introducing new bugs is a challenging task.



Readings

Chapter 6 to 10, Object-Oriented Reengineering Patterns by Serge Demeyer

TESTS: YOUR LIFE INSURANCE!

- Write Tests to Enable Evolution
- Grow Your Test Base Incrementally
- Use a Testing Framework
- Test the Interface, Not the Implementation
- Record Business Rules as Tests
- Write Tests to Understand

Detecting Duplicated Code

- Compare Code Mechanically
- Visualize Code as Dot plots

Redistribute Responsibilities

- Move Behavior Close to Data
- Eliminate Navigation Code
- Split Up GodClass

Migration Strategies

- Involve the Users
- Build Confidence
- Migrate Systems Incrementally
- Prototype the Target Solution
- Always Have a Running Version
- Regression Test After Every Change
- Make a Bridge to the New Town
- Present the Right Interface
- Distinguish Public from Published Interface
- Deprecate Obsolete Interfaces
- Conserve Familiarity
- Use Profiler Before Optimizing

Transform Conditionals to Polymorphism

- Transform Self Type Checks
- Transform Client Type Checks
- Transform Conditionals into Registration
- Factor out State
- Factor out Strategy
- Introduce Null Object

Write Tests to Enable Evolution

Protect your investment in the legacy code by imposing a systematic testing program.

Introduce a testing process based on tests that are automated, repeatable and stored.

Well-designed tests exhibit the following properties:

- **Automation**. Tests should run without human intervention.
- **Persistence**. Tests must be stored to be automatable. Each test documents its test data, the actions to perform, and the expected results.
- **Repeatability**. Confidence in the system is increased if tests can be repeated after any change is implemented. Whenever new functionality is added, new tests can be added to the pool of existing tests, thereby increasing the confidence in the system
- **Unit testing**. Tests should be associated to individual software components so that they identify clearly which part of the system they test
- **Independence.** Each test should minimize its dependencies on other tests. Dependent tests typically result in avalanche effects: when one test breaks, many others break as well.

TESTS: YOUR LIFE INSURANCE!

Grow Your Test Base Incrementally

Balance the costs and the benefits of tests by incrementally introducing just the tests you need at a given point in time.

Carefully assess your priorities and initially develop tests only for the most critical components. As you reengineer the system, intro duce tests for the new features, parts of the legacy that may be affected, and any bugs you identify along the way.

Use a Testing Framework

Encourage developers to write and use regression tests by providing a framework that makes it easy to develop, organize and run tests.

Use a testing framework that allows suites of tests to be composed from individual test cases.

TESTS: YOUR LIFE INSURANCE!

Test the Interface, Not the Implementation

Also Known As: Black-Box Testing

Build up reusable tests that focus on external behavior rather than on implementation details, and thereby will survive changes to the system.

- Legacy systems have many features that should continue to function as the system evolves.
- You cannot afford to spend too much time writing tests while reengineering the system. You do not want to waste effort in developing tests that will have to be changed as you change the system.

Record Business Rules as Tests

Keep the system in sync with the business rules it implements by encoding the rules explicitly as tests.

Write executable tests that record the business rules as test cases, actions, and tests over the results. When tests break, you know that things are out of sync.

Write Tests to Understand

Record your understanding of a piece of code in the form of executable tests, thus setting the stage for future changes.

- Tests can be applied to gain different levels of understanding. For example, black-box tests can help you to refine your understanding of roles and collaborations, whereas white-box tests can help you to gain understanding of the implementation of complex logic.
- The tests that you develop will help to enable future reengineering effort. Tests will force you to be precise about the creation and the use of the objects under test.

TESTS: YOUR LIFE INSURANCE!

Migration Strategies

- How can you be sure that the new system will be accepted by users?
- How do you migrate to the new system while the old system is being used?
- How can you test and evaluate the new system before it is finished?

It is not enough to reengineer a legacy system and then deploy it. In fact, if you try this, you will surely fail (for the same reasons that big Water fall projects in new territories often fail). You must be prepared to intro duce the new solution gradually, to gain the confidence and collaboration of the users, and you must adopt a strategy for migrating gradually and painlessly from the existing system, while it is still being deployed, to the new system.

Involve the Users

Also Known As: Engage Customers

Maximize acceptance of changes by involving the users at every step

Create an environment that will encourage contact between users and developers. Physical location is important.

Establish simple procedures for delivering intermediate results on a regular basis and obtaining feedback.

Early prototypes may help, especially to evaluate risky new technologies or approaches.

Build Confidence

Improve your chances of overall success by demonstrating results in regular increments.

Pick short intervals for delivering new results. At each step, try to agree together with the users what are the smallest results that can demonstrate real value.

By taking smaller steps, you reduce the risk that an individual step will fail.

Frequent, positive results help to build confidence.

Migrate Systems Incrementally

Also Known As: Chicken Little

Avoid complexity and risk of big-bang reengineering by deploying functionality in frequent increments

Deploy a first update of the legacy system as soon as you can, and migrate incrementally to the target system.

Steps

- Decompose the legacy system into parts.
- Choose one part to tackle at a time.
- Put tests in place for that part and the parts that depend on it
- Take appropriate steps to wrap, reengineer or replace the legacy component.
- Deploy the updated component and obtain feedback.
- Iterate.

Prototype the Target Solution

Evaluate the risk of migrating to a new target solution by building a prototype

Steps

- Identify the biggest technical risks for your reengineering project. Typically they will concern things like:

– choice of a new system architecture
– migration of legacy data to new system
– adequate performance — or performance gains — with new technology or platform (for example, demonstrating that a cer tain transaction throughput can be achieved)

- Decide whether to implement an **exploratory (i.e., throwaway)** pro totype that will service purely to evaluate the feasibility of a technical option, or rather an **evolutionary prototype** that will eventually evolve into the new target system

- The exploratory prototype does not need to be designed to address any other issues or questions, and will not be part of the migrated system. An evolutionary prototype, on the other hand, is intended to eventually replace a legacy component, and must therefore reflect the target architecture.

Always Have a Running Version

How do you convince your customer that you are on the right path?

This problem is difficult because:

- It can be hard to demo a software system under development, or to discuss problems with users since there is often no stable, running version of the system available.

- Integrating changes from multiple versions of a system can be slow and painful.

Institute a discipline of integrating new changes and developments on a daily basis.

Steps

- Have version management and configuration management systems in place.
- Make sure you have regression tests in place for the parts you are working on.
- Institute a discipline of short transactions for checking out system components and checking them back in again.
- Plan iterations to be as short as possible to allow changes to be integrated into a running system

Many practitioners advocate a process of continuous integration as a way to avoid a risky and painful big-bang integration

Regression Test After Every Change

Build confidence by making sure that whatever worked before still works

Run your regression test suite every time you think you have reached a stable state

If you consistently build up tests for defects you discover and new features, you will end up with a reusable test base that gives you confidence that your changes are sound, and helps you detect problems earlier.

Make a Bridge to the New Town

Migrate data from a legacy system by running the new system in parallel, with a bridge in between

Make a(data) bridge that will incrementally transfer data from the legacy system to the replacement system as new components are ready to take the data over from their legacy counterparts

Steps

- Identify legacy and replacement components that deal with the same logical data entities.
- Implement a “data bridge” which is responsible for redirecting read requests from the new component to the legacy data source, if the data have not already been migrated.
- The bridge is responsible for any necessary data conversion. The new component should not be aware of the bridge.
- Adapt the legacy component to redirect write requests to the new component, so that the new data stay up-to-date.
- When all the data have been transferred, remove the bridge and the legacy component.

Present the Right Interface

Also Known As: Semantic Wrapper

How should the new target system access legacy services during the migration process?

Identify the abstractions that you want to have in the new system, and wrap up the old software to emulate the new abstractions.

Present the Right Interface frees you from thinking in terms of the legacy design and makes it easier to consider alternative approaches.

Consider, **for example**, a procedural graphics library that will be used within an object-oriented system. It will be too costly and time-consuming to reimplement the library in an object-oriented way. It would be easier to wrap it as a utility class (i.e., as a class with static methods but no instances), but it would be wiser to write a slightly thicker wrapper that presents a truly object-oriented interface, but is implemented using the underlying procedural abstractions. In this way the new system will not be polluted by legacy abstractions.

Distinguish Public from Published Interface

Facilitate parallel development by distinguishing unstable “published interfaces” from stable “public interfaces”.

Since “published” interfaces are not supported by any programming language, you may have to use naming conventions, or abuse other features to achieve the desired effect.

In Java, consider declaring such interfaces as protected, or giving them package scope(undeclared). When the interfaces stabilize, you may redeclare them as being public.

In C++, consider declaring components with published interfaces private or protected, and declare as friends the clients that are permitted to use them. When the interfaces stabilize, redeclare the components as public, and delete the declarations of friends.

Deprecate Obsolete Interfaces

Give clients time to react to changes to public interfaces by flagging obsolete interfaces as “deprecated”.

Flag the old interface as being “deprecated”, thereby notifying clients that it will almost certainly be removed in the next upcoming release.

Steps

- You have determined that a public interface should be changed, but you do not want to break all clients. Implement the new interface, but “deprecate” the old one.
- The deprecation mechanism should inform clients that the interface has changed, and that a newer interface is recommended instead.
- Evaluate to what extent the deprecated interface continues to be used, and whether it can be permanently retired. Consider removing it in a future release.

Conserve Familiarity

The legacy system requires significant changes.
The users are not happy with the legacy system, but they understand it well.

- You can migrate incrementally to a new solution.
- Introduce only a constant, relatively low number of changes between each new release.
- Users do not have to change their work habits too much between releases.

Use Profiler Before Optimizing

- When you are reengineering software, you are likely to encounter many naive algorithms in the legacy code.
- It can be hard to predict what will impact performance, and you can lose a lot of time on pure supposition.
- Optimized code is often more complex than simple, naive code.

- Whenever you are tempted to optimize a “clearly inefficient” part of the system, first use a profiler to determine whether it is actually a bottleneck.
- Don’t optimize anything unless your profiler tells you it will make a difference.
- If you decide to go ahead, prepare benchmarks that will demonstrate the performance gains.
- You do not waste time optimizing something that will not make a difference to overall performance.

Detecting Duplicated Code

- Duplicated code hampers the introduction of changes, since every implemented variant of a piece of functionality will have to be changed.
- Since it is easy to miss some variants, bugs are likely to pop up in other places.
- Duplicated code replicates and scatters the logic of a system instead of grouping it into identifiable artifacts (classes, methods, packages).
- It leads to systems that are more difficult to understand and to change.

Detecting Duplicated Code consists of **two patterns**:

1. Compare Code Mechanically, which describes how we can detect duplicated code, and
2. Visualize Code as Dot plots, which shows how duplicated code can be better understood by simple matrix visualization.

Compare Code Mechanically

Discover duplicated code by comparing all the source code files line-by line.

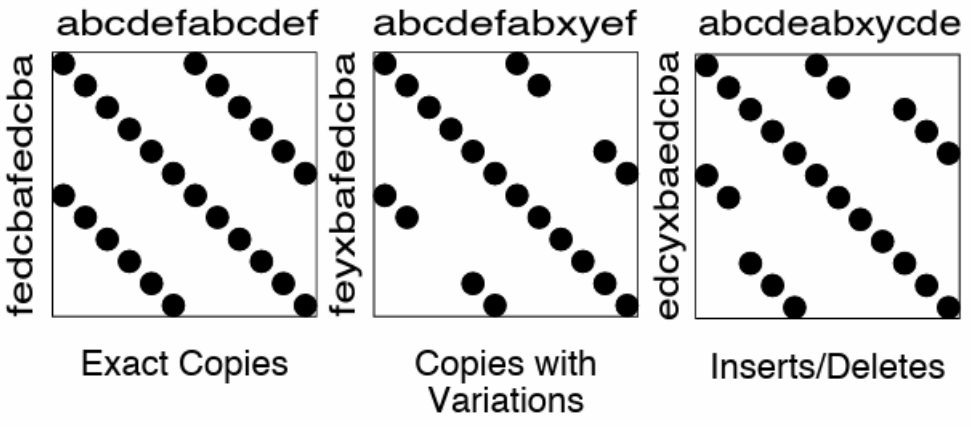
Steps

- Normalize the lines of code by removing comments, tabs and blanks.
- Remove lines that contain uninteresting code elements (e.g., just else or })
- Compare each line with all the other lines.
- Reduce search complexity by hashing:
- **Preprocessing:**
  - ❑ Compute the hash value for each line
  - ❑ Actual Comparison: Compare all lines in the same hash bucket

Visualize Code as Dot plots

Gain insight into the nature of the duplication by studying the patterns in the dot plots.

Visualize the code as a matrix in which the two axes represent two source code files (possibly the same file), and dots in the matrix occur where source code lines are duplicated.



Possible sequences of dot and their associated interpretations

Redistribute Responsibilities

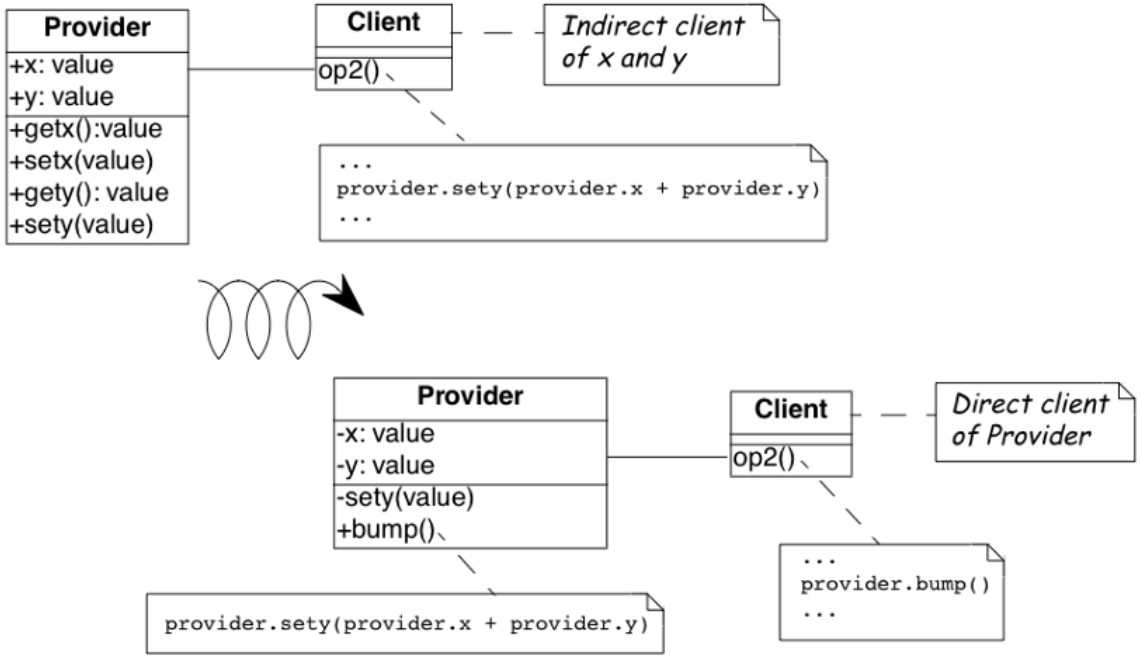
- This cluster deals with problems of misplaced responsibilities.
- The two extreme cases are **data containers**, classes that are nothing but glorified data structures and have almost no identifiable responsibilities, and **god classes**, procedural monsters that assume too many responsibilities.

Data containers (objects that just provide access to data, but no own behavior) are a simple and convenient way to share information between many subsystems.

God classes (big classes that do all of the work) are hard to extend, modify or subclass because such changes affect large numbers of other methods or instance variables.

Move Behavior Close to Data

Strengthen encapsulation by moving behavior from indirect clients to the class containing the data it operates on.



Classes that were mere data containers are transformed into real service providers

Redistribute Responsibilities

- Data containers are converted to service providers with clear responsibilities.
- The service providers become more useful to other clients.
- Clients are no longer responsible for implementing provider behavior.

It makes use of the refactorings **ExtractMethod** and **MoveMethod**, since the behavior in question will have to be extracted from a client method and then moved to a provider class.

1. Identify the client behavior that you want to move, i.e., the complete method or a part of a method that accesses provider data.
2. Create the corresponding method in the provider class, if it does not already exist.
3. Give an intention-revealing name to the new method.
4. In the client invoke the new provider method with the correct parameters.

Eliminate Navigation Code

Reduce the impact of changes by shifting responsibility down a chain of connected classes.

How do you reduce coupling due to classes that navigate through the object graph?

Changes in the interfaces of a class will affect not only direct clients, but also all the indirect clients that navigate to reach it.

Iteratively move behavior defined by an indirect client to the container of the data on which it operates.

Look for indirect clients that contain a lot of navigation code. Navigation code is of two kinds:

1. a sequence of attribute accesses, e.g.,a.b.c.d where b is an attribute of a, c is an attribute of b and d an attribute of c. The result of such a sequence can be assigned to variable or a method of the last object can be invoked, e.g.,a.b.c.d.op().
2. A sequence of accessor method calls. In Java and C++ such a sequence has the form object.m1().m2().m3() where object is an expression returning an object, m1 is a method of object, m2 a method of the object returned by the invocation of m1, m3 a method of the object returned by the invocation of m2 and so on.

Navigation code can be detected by simple pattern matching.

Split Up God Class

Split up a class with too many responsibilities into a number of smaller, cohesive classes.

- Evolution of the application is difficult because nearly every change touches this class, and affects multiple responsibilities.
- It is difficult to understand the different abstractions that are inter mixed in a god class. Most of the data of the multiple abstractions are accessed from different places.
- Identifying where to change a feature without impacting the other functionality or other objects in the system is difficult.
- Moreover, changes in other objects are likely to impact the god class, thus hampering the evolution of the system.
- It is nearly impossible to change a part of the behavior of a god class in a black-box way.

Pros: **Application control is no longer centralized in a single monolithic entity but distributed amongst entities that each assume a well defined set of responsibilities. The design evolves from a procedural design towards an object-oriented design based on autonomous interacting objects.**

Redistribute Responsibilities

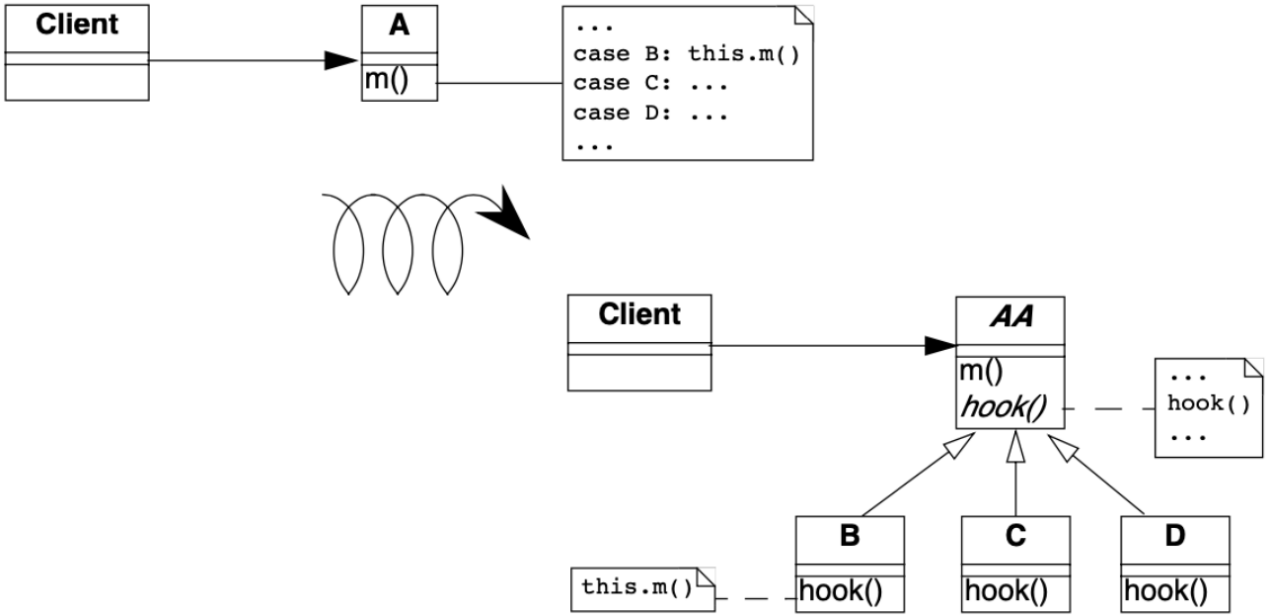
Transform Conditionals to Polymorphism

eliminate the large conditionals, thereby reducing coupling between classes, and improving flexibility in the face of future changes

Large conditionals are often a sign that behavior implemented by clients should probably be be shifted to the provider classes. Typically a new method will be introduced to the provider hierarchy, and the individual cases of the conditional statement will each move to one of the provider classes.

Transform Self Type Checks

Improve the extensibility of a class by replacing a complex conditional statement with a call to a hook method implemented by subclasses.



Transformation of explicit type check into self polymorphic method calls.

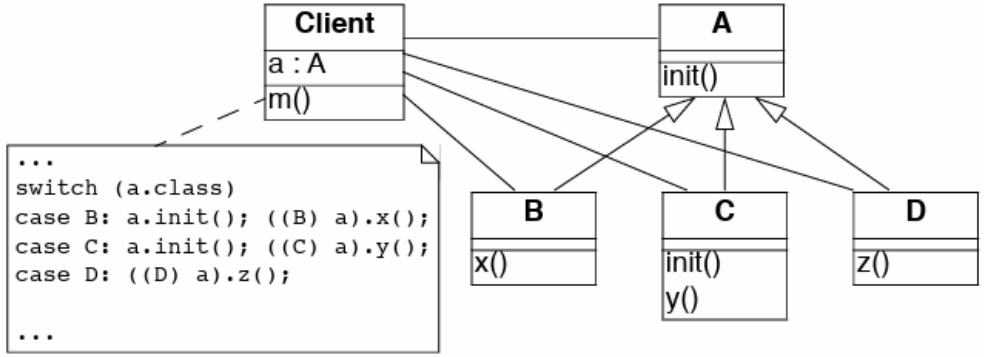
- New behaviors can now be added in a incremental manner, without having to change a set of methods of a single class containing all the behavior.
- A specific behavior can now be understood independently from the other variations.
- A new behavior represents its data independently from the others, thereby minimizing the possible interference and increasing the understandability of the separated behaviors.
- All behaviors now share a common interface, thereby improving their readability.

Transform Client Type Checks

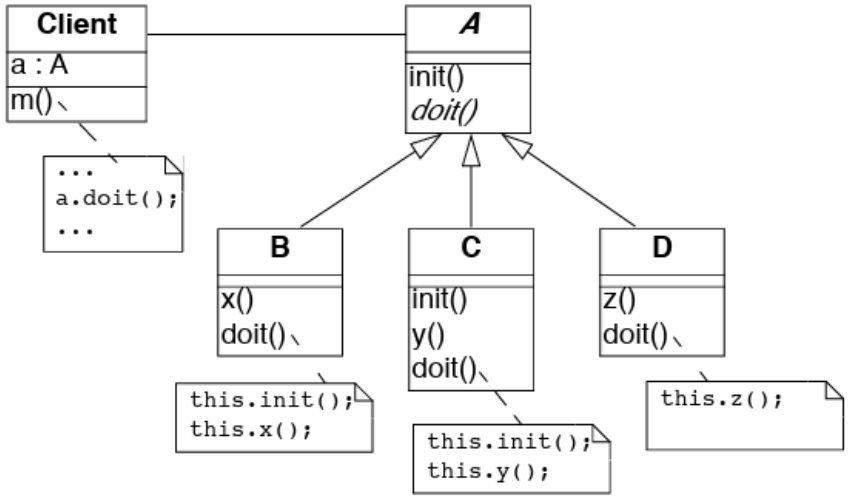
Reduce client/provider coupling by transforming conditional code that tests the type of the provider into a polymorphic call to a new provider method

How do you reduce the coupling between clients and providers of ser vices, where the clients explicitly check the type of providers and have the responsibility to compose providers code?

- Introduce a new method to the provider hierarchy.
- Implement the new method in each subclass of the provider hierarchy by moving the corresponding case of the clients conditional to that class.
- Replace the entire conditional in the client by a simple call to the new method.



Transformation of explicit type check used to determine which methods of a client should be invoked into polymorphic method calls.



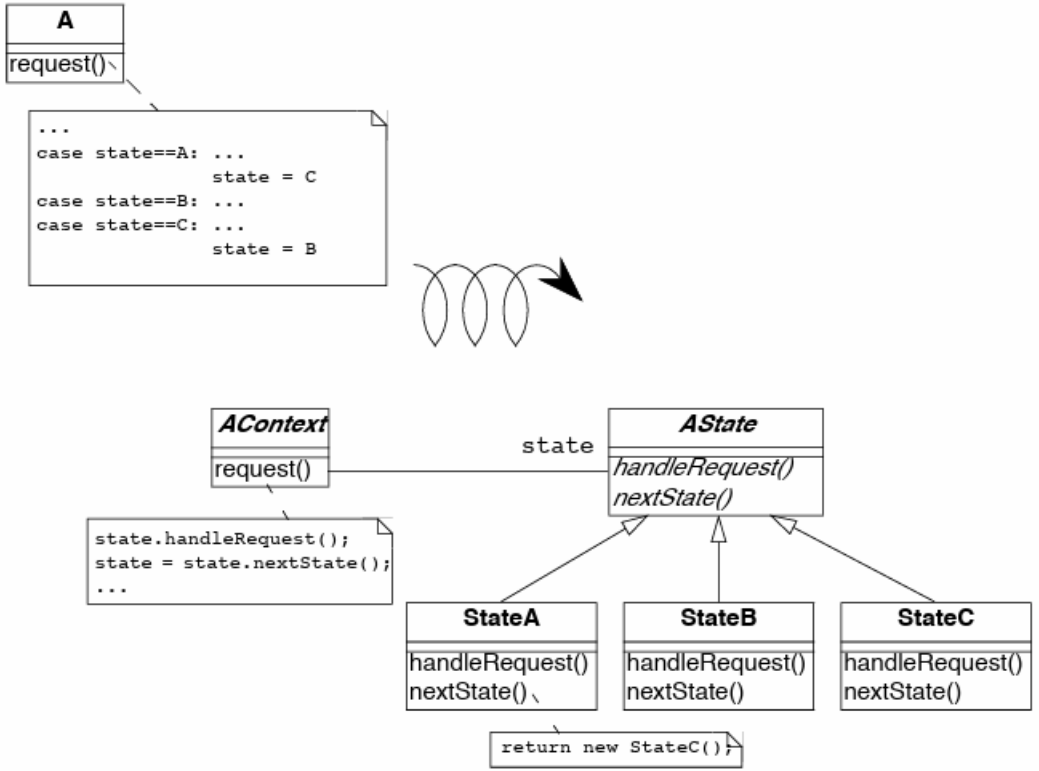
Factor out State

Eliminate complex conditional code over an object's state by applying the State design pattern.

- There are several complex conditional statements spread out over the methods of the object. Adding new behavior may affect these conditionals in subtle ways.
- Whenever new possible states are introduced, all the methods that test state have to be modified.

Apply the State pattern, i.e., encapsulate the state-dependent behavior into separate objects, delegate calls to these objects and keep the state of the object consistent by referring to the right instance of these state objects

Transformation to go from a state pattern simulated using explicit state conditional to a situation where the state pattern has been applied.

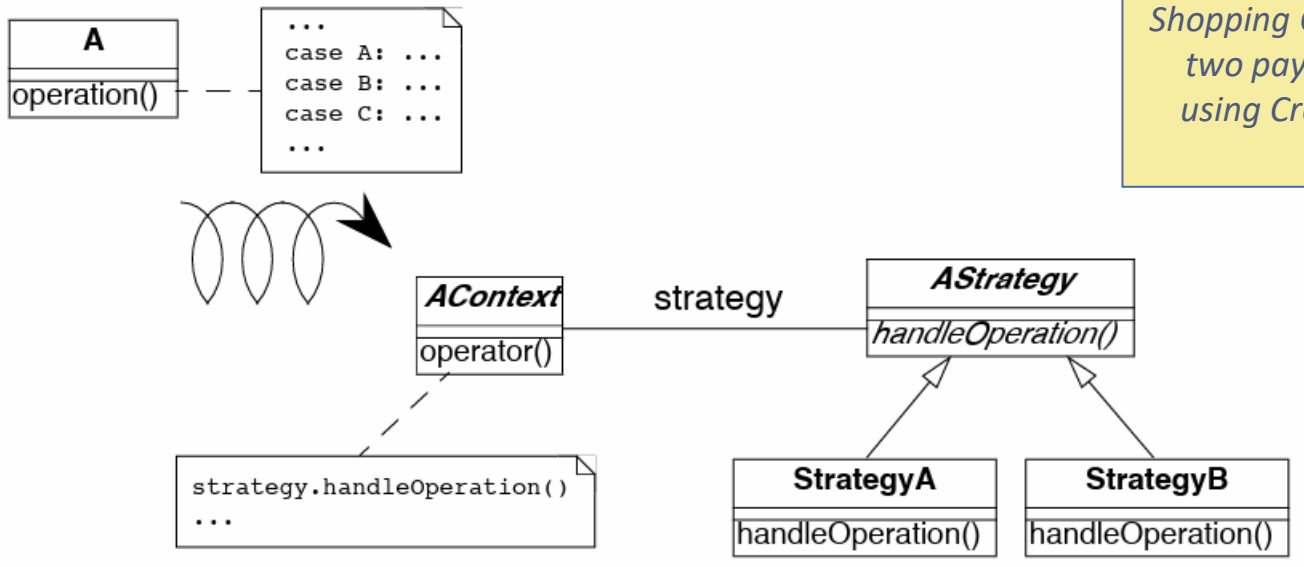


Transform Conditionals to Polymorphism

Factor out Strategy

Eliminate conditional code that selects a suitable algorithm by applying the Strategy design pattern.

Apply the Strategy pattern, i.e., Strategy pattern is used when we have multiple algorithm for a specific task and client decides the actual implementation to be used at runtime.



Shopping Cart where we have two payment strategies - using Credit Card or using PayPal.

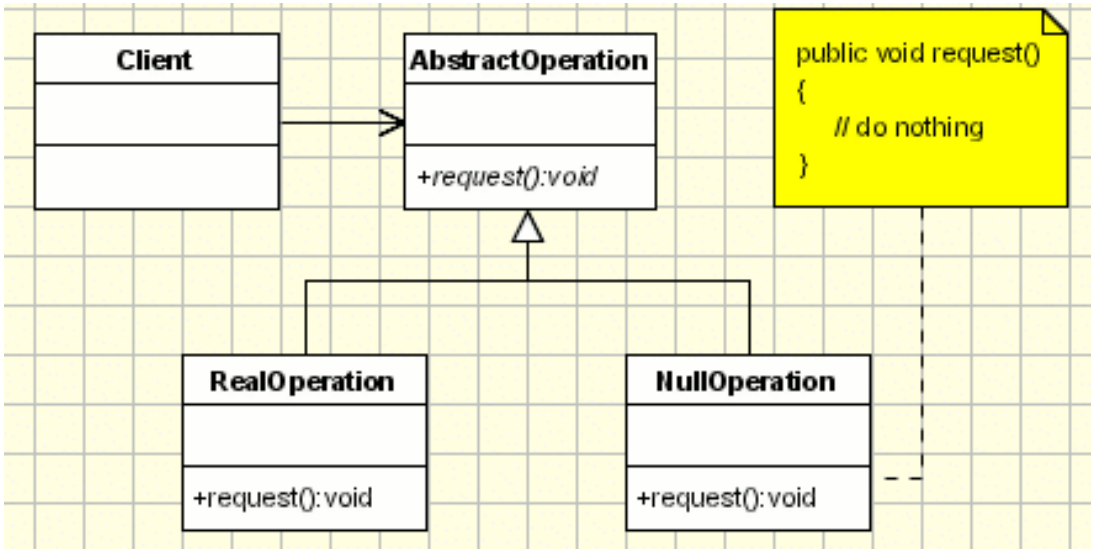
Introduce Null Object

Eliminate conditional code that tests for null values by applying the Null Object design pattern

- Client methods are always testing that certain values are not null before actually invoking their methods.
- Adding a new subclass to the client hierarchy requires testing null values before invoking some of the provider methods.

Apply the Null Object pattern, i.e., encapsulate the null behavior as a separate provider class so that the client class does not have to perform a null test.

The participants classes in this pattern are:
AbstractClass - defines abstract primitive operations that concrete implementations have to define.
RealClass - a real implementation of the AbstractClass performing some real actions.
NullClass - a implementation which do nothing of the abstract class, in order to provide a non-null object to the client.
Client - the client gets an implementation of the abstract class and uses it. It doesn't really care if the implementation is a null object or an real object since both of them are used in the same way.



Transform Conditionals into Registration

Improve the modularity of a system by replacing conditionals in clients by a registration mechanism.

- Having one single place to look for all the kinds of tools makes it easy to understand the system and easy to add new tools.
- However, every time you remove a tool, you have to remove one case in some conditional statement, else certain parts (tool clients) would still reflect the presence of the removed tools, leading to fragile systems.
- Then every time you add a new tool, you have to add a new conditional in all the tool clients.

Introduce a registration mechanism to which each tool is responsible for registering itself, and transform the tool clients to query the registration repository instead of performing conditionals.

That’s it